

Theoretical Framework

Genetic Algorithms (GAs) are a class of optimization techniques inspired by the principles of biological evolution. They are used to find approximate solutions to complex problems by mimicking the process of natural selection. In a GA, individuals are represented as chromosomes, which encode potential solutions to the problem at hand. These chromosomes evolve over generations through genetic operators, such as selection, crossover, and mutation. The fitness of individuals is assessed based on an objective function, and the algorithm seeks to improve the population over successive generations.

Representation of Chromosomes:

In the provided codes, chromosomes are represented as sequences of bits, with the length of these chromosomes determined by the variable `L_chromosome`. The values of the genes are decoded into variables in the search space, such as `x`, `y`, and `z` for the Rastrigin function, or `x` and `y` for the Ackley function. This binary encoding allows the algorithm to explore and optimize solutions.

Objective Functions:

The success of a GA depends on the choice of the objective function. The codes implement two popular objective functions:

Ackley Function:

The Ackley function is a classic test function in optimization. It is characterized by its smooth, two-variable landscape. The function has a global minimum at the origin $(0, 0)$ and multiple local minima. Its shape poses challenges for optimization algorithms due to the presence of these local optima. The GA's goal in this case is to find the global minimum by evolving a population of chromosomes.

Rastrigin Function:

The Rastrigin function is another widely used objective function in optimization. It is particularly useful for testing algorithms in high-dimensional search spaces. Similar to the Ackley function, the Rastrigin function has a global minimum at the origin $(0, 0, 0)$ and numerous local minima. GAs aim to explore and converge to the global minimum within this complex landscape.

Material and equipment

The computational environment utilized for this project is Google Colab (Google Colaboratory). Google Colab is a cloud-based Python development platform that offers free access to a virtual environment for running Python code, executing data analysis, and performing machine learning tasks.

Notebook Setup

- Execution environment type: Python 3
- Hardware accelerator: CPU

Operating System:

- Distributor ID: Ubuntu
- Description: Ubuntu 22.04.2 LTS (Long Term Support)
- Release: 22.04
- Codename: jammy

CPU Information:

- Architecture: x86_64
- CPU op-mode(s): 32-bit, 64-bit
- CPU(s): 2
- Vendor ID: GenuineIntel
 - Model name: Intel(R) Xeon(R) CPU @ 2.20GHz

Practice development

Write GA's to minimize:

- Ackley's function in 2 dimensions.

The code is an implementation of a Genetic Algorithm (GA) to optimize the Ackley function, which is a widely used benchmark problem in optimization. The Ackley function is characterized by having a global minimum at the origin (0, 0) and multiple local minima, making it a challenging landscape for optimization algorithms. In this GA, chromosomes are represented as binary strings of a fixed length, and individuals are evolved over generations through selection, crossover, and mutation. Elitism ensures that the best-performing individuals are preserved, and the algorithm aims to find the global minimum of the Ackley function by evolving the population. Users can interact with the code to specify the number of generations for the algorithm to run, allowing for experimentation and observation of its convergence behavior.

This code serves as a practical demonstration of how GAs can be applied to tackle complex optimization problems, providing insights into the optimization landscape and the interplay between genetic operators and function evaluation.

```
#Ackley Function

import math
import random
import time
import matplotlib.pyplot as plt
import numpy as np
from IPython import display as display
from functools import cmp_to_key

#Chromosomes are 10 bits long
L_chromosome=30
N_chains=2**L_chromosome
#Lower and upper limits of search space
a=-5
b=5
crossover_point=int(L_chromosome/2)

def random_chromosome2(L_chromosome):
    chromosome=[]
    for i in range(0,L_chromosome):
        if random.random()<0.5:
            chromosome.append(0)
        else:
            chromosome.append(1)
    return chromosome
```

```

#Number of chromosomes
N_chromosomes=30
#probability of mutation
probab_m=0.5

F0=[]
fitness_values=[]

for i in range(0,N_chromosomes):
    F0.append(random_chromosome2(L_chromosome))
    fitness_values.append(0)

#binary codification (two variables)
def decode_chromosome2(chromosome):
    L_gen=len(chromosome)//2
    N_chains=2**L_gen
    gen1=chromosome[:L_gen]
    gen2=chromosome[L_gen:]
    value1=0
    value2=0
    for p in range(L_gen):
        value1+=(2**p)*gen1[-1-p]
        value2+=(2**p)*gen2[-1-p]

    x=a+(b-a)*float(value1)/(N_chains-1)
    y=a+(b-a)*float(value2)/(N_chains-1)
    return x,y

def f(x,y):
    return (-20.0*math.exp(-0.2 * math.sqrt(0.5 * (pow(x,2) +
pow(y,2))))
    -math.exp(0.5 * (math.cos(2 * math.pi * x)+math.cos(2 * math.pi *
y)))) + math.e + 20)

def evaluate_chromosomes():
    global F0
    for p in range(N_chromosomes):
        vx,vy=decode_chromosome2(F0[p])
        fitness_values[p]=f(vx,vy)

def compare_chromosomes(chromosome1,chromosome2):
    vc1x,vc1y=decode_chromosome2(chromosome1)
    vc2x,vc2y=decode_chromosome2(chromosome2)
    fvc1=f(vc1x,vc1y)

```

```

    fvc2=f(vc2x,vc2y)
    if fvc1 > fvc2:
        return 1
    elif fvc1 == fvc2:
        return 0
    else: #fvg1<fvg2
        return -1

suma=float(N_chromosomes*(N_chromosomes+1))/2.

Lwheel=N_chromosomes*10

def create_wheel():
    global F0,fitness_values
    maxv=max(fitness_values)
    acc=0
    for p in range(N_chromosomes):
        acc+=maxv-fitness_values[p]
    fraction=[]
    for p in range(N_chromosomes):
        fraction.append( float(maxv-fitness_values[p])/acc)
        if fraction[-1]<=1.0/Lwheel:
            fraction[-1]=1.0/Lwheel
    ##    print fraction
    fraction[0]==(sum(fraction)-1.0)/2
    fraction[1]==(sum(fraction)-1.0)/2
    ##    print fraction
    wheel=[]
    pc=0
    for f in fraction:
        Np=int(f*Lwheel)
        for i in range(Np):
            wheel.append(pc)
        pc+=1
    return wheel

F1=F0[:]

def nextgeneration(b):
    F0.sort(key=cmp_to_key(compare_chromosomes))
    print( "Best solution so far:")
    x,y=decode_chromosome2(F0[0])
    print( "f(",x,"",y,"")= ", f(x,y))
    #elitism, the two best chromosomes go directly to the next
    generation

```

```

F1[0]=F0[0]
F1[1]=F0[1]
for i in range(0,int((N_chromosomes-2)/2)):
    roulette=create_wheel()
    #print("r: ",roulette)
    #Two parents are selected
    p1=random.choice(roulette)
    p2=random.choice(roulette)
    #Two descendants are generated
    o1=F0[p1][0:crossover_point]
    o1.extend(F0[p2][crossover_point:L_chromosome])
    o2=F0[p2][0:crossover_point]
    o2.extend(F0[p1][crossover_point:L_chromosome])
    #Each descendant is mutated with probability prob_m
    if random.random() < prob_m:
        o1[int(round(random.random()*(L_chromosome-1)))]^=1
    if random.random() < prob_m:
        o2[int(round(random.random()*(L_chromosome-1)))]^=1
    #The descendants are added to F1
    F1[2+2*i]=o1
    F1[3+2*i]=o2

    #The generation replaces the old one
    F0[:]=F1[:]

F0.sort(key=cmp_to_key(compare_chromosomes))
evaluate_chromosomes()

# Función para ejecutar la evolución durante un número especificado de
generaciones
def evolve_generations(num_generations):
    for _ in range(num_generations):
        nextgeneration(None) # Simulamos un clic en el botón

# Bucle interactivo para ejecutar la evolución
while True:
    user_input = input("Ingrese el número de generaciones a
evolucionar (o 'exit' para salir): ")

    if user_input.lower() == "exit":
        break # Salir del programa si se ingresa "exit"

    try:
        num_generations = int(user_input)
        evolve_generations(num_generations)

```

```

    except ValueError:
        print("Entrada inválida. Por favor, ingrese un número o 'exit' para salir.")

# Si llega a este punto, el programa ha terminado
print("Programa finalizado.")

```

- Rastrigin's function in 3 dimensionsKnapsack Problem

The provided code implements a Genetic Algorithm (GA) to optimize the Rastrigin function, a well-known benchmark problem in optimization. The Rastrigin function is especially useful for testing optimization algorithms in high-dimensional search spaces. It features a global minimum at the origin (0, 0, 0) and numerous local minima, making it a complex landscape for optimization.

In this GA, chromosomes are represented as binary strings of fixed length, and individuals are evolved over generations using selection, crossover, and mutation. Elitism is employed to retain the best-performing individuals. The objective is to explore and converge towards the global minimum of the Rastrigin function by evolving the population.

User interaction allows the user to specify the number of generations for the algorithm to run, facilitating experimentation and observation of convergence behavior.

This code demonstrates the application of GAs to challenging optimization problems, providing insights into navigating high-dimensional spaces and dealing with complex landscapes with multiple local optima.

```

#Rastrigin Function

import math
import random
import time
import matplotlib.pyplot as plt
import numpy as np
from IPython import display as display
from functools import cmp_to_key

#Chromosomes are 10 bits long
L_chromosome=210
N_chains=2**L_chromosome
#Lower and upper limits of search space
a=-5.12
b=5.12
crossover_point=int(L_chromosome/2)
num_gen=0

```

```

def random_chromosome3(L_chromosome):
    chromosome=[]
    for i in range(0,L_chromosome):
        if random.random()<0.1:
            chromosome.append(0)
        else:
            chromosome.append(1)
    return chromosome

#Number of chromosomes
N_chromosomes=100
#probability of mutation
probab_m=0.5

F0=[]
fitness_values=[]

for i in range(0,N_chromosomes):
    F0.append(random_chromosome3(L_chromosome))
    fitness_values.append(0)

#binary codification (three variables)
def decode_chromosome3(chromosome):
    L_gen=len(chromosome)//3
    N_chains=2**L_gen
    gen1=chromosome[:L_gen]
    gen2=chromosome[L_gen:2*L_gen]
    gen3=chromosome[2*L_gen:]
    value1=0
    value2=0
    value3=0
    for p in range(L_gen):
        value1+=(2**p)*gen1[-1-p]
        value2+=(2**p)*gen2[-1-p]
        value3+=(2**p)*gen3[-1-p]

    x=a+(b-a)*float(value1)/(N_chains-1)
    y=a+(b-a)*float(value2)/(N_chains-1)
    z=a+(b-a)*float(value3)/(N_chains-1)
    return x,y,z

def f(x,y,z):
    return (10**3+(x**2-10*math.cos(2*math.pi*x))+(y**2-
10*math.cos(2*math.pi*y))+(z**2-10*math.cos(2*math.pi*z)))

```



```

def evaluate_chromosomes():
    global F0
    for p in range(N_chromosomes):
        vx,vy,vz=decode_chromosome3(F0[p])
        fitness_values[p]=f(vx,vy,vz)

def compare_chromosomes(chromosome1,chromosome2):
    vc1x,vc1y,vc1z=decode_chromosome3(chromosome1)
    vc2x,vc2y,vc2z=decode_chromosome3(chromosome2)
    fvc1=f(vc1x,vc1y,vc1z)
    fvc2=f(vc2x,vc2y,vc2z)
    if fvc1 > fvc2:
        return 1
    elif fvc1 == fvc2:
        return 0
    else: #fvg1<fvg2
        return -1

suma=float(N_chromosomes*(N_chromosomes+1))/2.

Lwheel=N_chromosomes*10

def create_wheel():
    global F0,fitness_values
    maxv=max(fitness_values)
    acc=0
    for p in range(N_chromosomes):
        acc+=maxv-fitness_values[p]
    fraction=[]
    for p in range(N_chromosomes):
        fraction.append( float(maxv-fitness_values[p])/acc)
        if fraction[-1]<=1.0/Lwheel:
            fraction[-1]=1.0/Lwheel
    ##    print fraction
    fraction[0]--=(sum(fraction)-1.0)/2
    fraction[1]--=(sum(fraction)-1.0)/2
    ##    print fraction
    wheel=[]
    pc=0
    for f in fraction:
        Np=int(f*Lwheel)
        for i in range(Np):
            wheel.append(pc)
        pc+=1
    return wheel

```

```

F1=F0[:]

def nextgeneration(b):
    global num_gen
    F0.sort(key=cmp_to_key(compare_chromosomes))
    print("Generation:",num_gen)
    num_gen+=1
    print( "Best solution so far:")
    x,y,z=decode_chromosome3(F0[0])
    print( "f(",x,"",y,"",z,"")= ", f(x,y,z))
    #elitism, the two best chromosomes go directly to the next
generation
    F1[0]=F0[0]
    F1[1]=F0[1]
    for i in range(0,int((N_chromosomes-2)/2)):
        roulette=create_wheel()
        #print("r: ",roulette)
        #Two parents are selected
        p1=random.choice(roulette)
        p2=random.choice(roulette)
        #Two descendants are generated
        o1=F0[p1][0:crossover_point]
        o1.extend(F0[p2][crossover_point:L_chromosome])
        o2=F0[p2][0:crossover_point]
        o2.extend(F0[p1][crossover_point:L_chromosome])
        #Each descendant is mutated with probability prob_m
        if random.random() < prob_m:
            o1[int(round(random.random()*(L_chromosome-1)))]^=1
        if random.random() < prob_m:
            o2[int(round(random.random()*(L_chromosome-1)))]^=1
        #The descendants are added to F1
        F1[2+2*i]=o1
        F1[3+2*i]=o2

    #The generation replaces the old one
    F0[:]=F1[:]

F0.sort(key=cmp_to_key(compare_chromosomes))
evaluate_chromosomes()

# Función para ejecutar la evolución durante un número especificado de
generaciones
def evolve_generations(num_generations):
    for _ in range(num_generations):

```

```

        nextgeneration(None)  # Simulamos un clic en el botón

# Bucle interactivo para ejecutar la evolución
while True:
    user_input = input("Ingrese el número de generaciones a
evolucionar (o 'exit' para salir): ")

    if user_input.lower() == "exit":
        break  # Salir del programa si se ingresa "exit"

    try:
        num_generations = int(user_input)
        evolve_generations(num_generations)
    except ValueError:
        print("Entrada inválida. Por favor, ingrese un número o 'exit'
para salir.")

# Si llega a este punto, el programa ha terminado
print("Programa finalizado.")

```

Conclusions and recommendations

In our exploration of Genetic Algorithms (GAs) using the Ackley and Rastrigin functions, it is evident that GAs are a potent tool for tackling complex optimization problems. Their ability to navigate landscapes with multiple local optima, adapt to different scenarios, and converge to global minima is noteworthy. However, we've also recognized the critical importance of parameter tuning, as GAs' performance is highly sensitive to factors such as population size, mutation rates, and the number of generations. Moreover, the interactive specification of the number of generations provides valuable insights into the algorithm's evolutionary progress, aiding in fine-tuning and comprehension of convergence trends.

To further maximize the potential of GAs, we recommend experimenting with diverse objective functions to gauge their adaptability to various optimization landscapes. Additionally, optimizing algorithm parameters, visualizing the convergence process, and testing GAs on real-world problems can enhance their effectiveness and applicability. Understanding the problem's characteristics and tuning parameters are key to achieving optimal results when employing a GA in optimization tasks.

References

- Google Colaboratory. (n.d.). <https://colab.research.google.com/>
- Python Software Foundation. (2023). Python Documentation. <https://docs.python.org/3>
- Matplotlib documentation — Matplotlib 3.7.2 documentation. (n.d.). <https://matplotlib.org/stable/index.html>
- NumPy documentation — NumPy v1.25 Manual. (n.d.). <https://numpy.org/doc/stable/>
- Sonja Surjanovic & Dereck Bingham, Virtual Library of Simulation Experiments: Test Functions and Datasets (n.d.). <https://www.sfu.ca/~ssurjano/ackley.html>